INFOSYS JAVA CODING PRACTICE

20 progressively harder problems • Offline-friendly solutions

Target date: 29 October 2026

Important: Candidate experiences are self-reported and vary by role, location, and assessment round. This is not an official Infosys paper. “Reported pattern” means a topic or problem family described by candidates; the samples and Java solutions here are reconstructed practice versions, not claimed verbatim test questions.

How to use: Attempt each problem for 15–30 minutes before reading the hint and solution. Run edge cases. Compile each method inside a class with a main method, or adapt it to the platform’s required signature.

A practical 8-day plan

Date	Focus	Problems
21 Oct	Number logic	1–4
22 Oct	Arrays and sets	5–8, 12–13, 15
23 Oct	Binary search and sorting	9, 14
24 Oct	Basic dynamic programming	10–11, 17
25 Oct	DP patterns	16, 19–20
26 Oct	Graphs and mixed review	18 + redo weak problems
27 Oct	Timed mock	Pick 3 problems: easy, medium, hard
28 Oct	Review only	Re-solve mistakes; sleep well

Problems, hints and solutions

1. 1. Reverse a number

Status: Core pattern; reconstructed practice

Problem: Reverse the digits of a non-negative integer.

Sample: 12340 **Expected:** 4321

Hint: Use % 10 to get the last digit and / 10 to remove it.

Java solution / core method:

```
static int reverse(int n) {
    int r = 0;
    while (n > 0) {
        r = r * 10 + n % 10;
        n /= 10;
    }
    return r;
}
```

Complexity: O(number of digits)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

2. 2. Palindrome number

Status: Core pattern; reconstructed practice

Problem: Return true if the integer reads the same forwards and backwards.

Sample: 1221 **Expected:** YES

Hint: Save the original number, reverse a copy, and compare.

Java solution / core method:

```
static boolean isPalindrome(int n) {
    int original = n, r = 0;
    while (n > 0) {
        r = r * 10 + n % 10;
        n /= 10;
    }
    return original == r;
}
```

Complexity: O(number of digits)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

3. 3. Prime check

Status: Core pattern; reconstructed practice

Problem: Print YES if n is prime, otherwise NO.

Sample: 29 **Expected:** YES

Hint: Check divisors only while i*i <= n; values below 2 are not prime.

Java solution / core method:

```
static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
```

Complexity: O(√n)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

4. 4. GCD of two numbers

Status: Core pattern; reconstructed practice

Problem: Find the greatest common divisor of 18 and 24.

Sample: 18 24 **Expected:** 6

Hint: Use Euclid's algorithm: repeatedly replace (a,b) with (b,a%b).

Java solution / core method:

```
static int gcd(int a, int b) {
    while (b != 0) {
        int rem = a % b;
        a = b;
        b = rem;
    }
    return a;
}
```

Complexity: O(log(min(a,b)))

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

5. 5. Second-largest distinct element

Status: Core pattern; common array pattern

Problem: Find the second-largest distinct value, or return NONE.

Sample: [4, 9, 9, 2, 7, 7] **Expected:** 7

Hint: Track largest and second largest; ignore duplicates of the largest.

Java solution / core method:

```
static Integer secondLargest(int[] a) {
    Integer first = null, second = null;
    for (int x : a) {
        if (first == null || x > first) {
            if (first == null || x != first) second = first;
            first = x;
        } else if (x != first &&
                   (second == null || x > second)) {
            second = x;
        }
    }
    return second;
}
```

Complexity: O(n) time, O(1) extra space

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

6. 6. Remove duplicates, preserve order

Status: Core pattern; reconstructed practice

Problem: Return each value only at its first appearance.

Sample: [1, 2, 1, 3, 2, 4] **Expected:** [1, 2, 3, 4]

Hint: A LinkedHashSet preserves insertion order.

Java solution / core method:

```
static int[] unique(int[] a) {
    java.util.LinkedHashSet<Integer> set =
        new java.util.LinkedHashSet<>();
    for (int x : a) set.add(x);
    int[] out = new int[set.size()];
    int i = 0;
    for (int x : set) out[i++] = x;
    return out;
}
```

Complexity: O(n) expected

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

7. 7. Anagram check

Status: Core pattern; reconstructed practice

Problem: Check whether two strings have the same character counts, ignoring case.

Sample: listen / silent **Expected:** YES

Hint: Normalize case, then compare sorted characters or frequency counts.

Java solution / core method:

```
static boolean anagram(String a, String b) {
    char[] x = a.toLowerCase().toCharArray();
    char[] y = b.toLowerCase().toCharArray();
    if (x.length != y.length) return false;
    java.util.Arrays.sort(x);
    java.util.Arrays.sort(y);
    return java.util.Arrays.equals(x, y);
}
```

Complexity: O(n log n)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

8. 8. Binary search

Status: Core pattern; standard assessment practice

Problem: Find target's index in a sorted ascending array, or -1.

Sample: [1,3,5,7,9], target=7 **Expected:** 3

Hint: Maintain low and high; discard half of the search range each iteration.

Java solution / core method:

```
static int binarySearch(int[] a, int target) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Complexity: O(log n)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

9. 9. Maximum in a mountain array

Status: Reported question pattern; reconstructed example

Problem: An array rises strictly then falls strictly. Find its peak value.

Sample: [1,3,7,12,9,5,2] **Expected:** 12

Hint: Compare a[mid] with a[mid+1] to choose which side contains the peak.

Java solution / core method:

```
static int peak(int[] a) {
    int low = 0, high = a.length - 1;
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (a[mid] < a[mid + 1]) low = mid + 1;
        else high = mid;
    }
    return a[low];
}
```

Complexity: O(log n)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

10. 10. Climbing stairs

Status: Reported interview pattern; reconstructed example

Problem: You may climb 1 or 2 steps. Count ways to reach step n.

Sample: n=5 **Expected:** 8

Hint: dp[i] = dp[i-1] + dp[i-2], with one way for 1 and two ways for 2.

Java solution / core method:

```
static long climb(int n) {
    if (n <= 2) return n;
    long a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        long c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

Complexity: O(n) time, O(1) space

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

11. 11. Maximum subarray sum

Status: Standard algorithm pattern; reconstructed example

Problem: Find the largest sum of any non-empty contiguous subarray.

Sample: [-2,1,-3,4,-1,2,1,-5,4] **Expected:** 6

Hint: Kadane's algorithm: extend the current run or start a new one at x.

Java solution / core method:

```
static int maxSubarray(int[] a) {
    int current = a[0], best = a[0];
    for (int i = 1; i < a.length; i++) {
        current = Math.max(a[i], current + a[i]);
        best = Math.max(best, current);
    }
    return best;
}
```

Complexity: O(n)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

12. 12. Two-sum existence

Status: Standard array/hash pattern; reconstructed example

Problem: Determine if two different elements sum to target.

Sample: [2,7,11,15], target=9 **Expected:** YES

Hint: As you scan x, check whether target-x has already been seen.

Java solution / core method:

```
static boolean twoSum(int[] a, int target) {
    java.util.HashSet<Integer> seen = new java.util.HashSet<>();
    for (int x : a) {
        if (seen.contains(target - x)) return true;
        seen.add(x);
    }
    return false;
}
```

Complexity: O(n) expected

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

13. 13. Rotate array right by k

Status: Common array pattern; reconstructed practice

Problem: Rotate an array right by k positions.

Sample: [1,2,3,4,5], k=2 **Expected:** [4,5,1,2,3]

Hint: Normalize k with k % n; use reversal or a temporary array.

Java solution / core method:

```
static int[] rotateRight(int[] a, int k) {
    int n = a.length;
    if (n == 0) return a;
    k %= n;
    int[] out = new int[n];
    for (int i = 0; i < n; i++)
        out[(i + k) % n] = a[i];
    return out;
}
```

Complexity: O(n) time, O(n) space

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

14. 14. Merge sort

Status: Reported interview pattern; reconstructed example

Problem: Sort an array using divide and conquer.

Sample: [8,3,5,1,9,2] **Expected:** [1,2,3,5,8,9]

Hint: Split into halves, sort each half recursively, then merge.

Java solution / core method:

```
static void mergeSort(int[] a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    int[] tmp = new int[r - l + 1];
    int i = l, j = m + 1, k = 0;
    while (i <= m && j <= r)
        tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
    while (i <= m) tmp[k++] = a[i++];
    while (j <= r) tmp[k++] = a[j++];
    System.arraycopy(tmp, 0, a, l, tmp.length);
}
```

Complexity: O(n log n)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

15. 15. Missing number from 1..n

Status: Core array pattern; reconstructed practice

Problem: Distinct numbers from 1..n have one missing. Find it.

Sample: n=5, [1,2,4,5] **Expected:** 3

Hint: Expected sum n(n+1)/2 minus actual sum. Use long for safety.

Java solution / core method:

```
static long missing(int n, int[] a) {
    long expected = (long)n * (n + 1) / 2;
    long actual = 0;
    for (int x : a) actual += x;
    return expected - actual;
}
```

Complexity: O(n)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

16. 16. Longest increasing subsequence length

Status: DP pattern; reconstructed practice

Problem: Find the length of the longest strictly increasing subsequence.

Sample: [10,9,2,5,3,7,101,18] **Expected:** 4

Hint: Use patience-sorting tails: binary-search the first tail >= x.

Java solution / core method:

```
static int lis(int[] a) {
    int[] tails = new int[a.length];
    int size = 0;
    for (int x : a) {
        int l = 0, r = size;
        while (l < r) {
            int m = (l + r) / 2;
            if (tails[m] < x) l = m + 1;
            else r = m;
        }
        tails[l] = x;
        if (l == size) size++;
    }
    return size;
}
```

Complexity: O(n log n)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

17. 17. House robber

Status: Reported interview pattern; reconstructed example

Problem: Choose non-adjacent values to maximize total.

Sample: [2,7,9,3,1] **Expected:** 12

Hint: At each house, choose between skipping it or adding it to the best result two houses back.

Java solution / core method:

```
static int rob(int[] a) {
    int prev2 = 0, prev1 = 0;
    for (int x : a) {
        int cur = Math.max(prev1, prev2 + x);
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

Complexity: O(n)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

18. 18. Count connected components

Status: Reported graph pattern; reconstructed example

Problem: Count connected groups in an undirected graph.

Sample: n=5; edges (0,1),(1,2),(3,4) **Expected:** 2

Hint: Build an adjacency list and run DFS/BFS from each unvisited vertex.

Java solution / core method:

```
static void dfs(int u, java.util.List<Integer>[] g,
                boolean[] seen) {
    seen[u] = true;
    for (int v : g[u])
        if (!seen[v]) dfs(v, g, seen);
}
// For each vertex: if not seen, components++;
// then call dfs(vertex, graph, seen).
```

Complexity: O(V+E)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

19. 19. Grid path DP

Status: Reported pattern family; reconstructed practice

Problem: Move from top-left to bottom-right using only right/down moves. Count paths in an obstacle-free grid.

Sample: rows=3, cols=3 **Expected:** 6

Hint: dp[r][c] = paths from above + paths from left; first row/column have one path.

Java solution / core method:

```
static long paths(int rows, int cols) {
    long[] dp = new long[cols];
    java.util.Arrays.fill(dp, 1);
    for (int r = 1; r < rows; r++)
        for (int c = 1; c < cols; c++)
            dp[c] += dp[c - 1];
    return dp[cols - 1];
}
```

Complexity: O(rows*cols) time, O(cols) space

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

20. 20. Target-sum sequences without equal neighbours

Status: Reported 2026 pattern family; reconstructed simplified version

Problem: Given allowed values {1,2,3}, count sequences summing to target where adjacent values cannot be equal.

Sample: target=4 **Expected:** 3

Hint: Track dp[sum][last value]; append a value only if it differs from the previous value. This is a small state-DP exercise.

Java solution / core method:

```
// dp[s][v] = ways to make sum s ending in v (v=1..3)
// dp[v][v] = 1 for the one-element sequence [v]
// For s from 1 to target, for each v in 1..3:
//   if (s == v) add the one-element sequence [v]
//   for each previous last u != v:
//       if (s-v >= 1) dp[s][v] += dp[s-v][u]
// Answer = sum_v dp[target][v].
// Be careful not to double-count one-element sequences.
```

Complexity: O(target * 3 * 3)

Test next: Try the smallest valid input, duplicates where relevant, and a case where no answer exists.

Final test-day checklist

- Read constraints before choosing an algorithm.
- Use long when sums or products may exceed int.
- Test empty/single-element arrays if allowed, duplicates, negatives, and boundary values.
- Do not print extra prompts like 'Enter number:' on an auto-graded platform.
- Compile early; check imports and exact output format.
- If stuck, write a brute-force solution first, then optimize.
- During the assessment, secure the easy problem before spending too long on the hardest one.